# Step 2 — A Real SQLite-Backed Store, and the Search Tool — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

**Story — Step 2:** a server that forgets everything on restart isn't something a real client could ever rely on, and a tool that silently stores an empty note is exactly the "fail loudly, not silently" bug Doc01 and Doc06 both exist to prevent. This step replaces the in-memory list with real SQLite storage, and adds a second Tool ( `search_notes` ) plus real content validation. **If not:** every demo would start from an empty store, and `add_note("")` would quietly create blank, useless rows.

After this step the project runs: notes survive a full restart, and empty text is rejected with a clear message. Files not shown here don't change.

## The store and the custom error

**Story — `exceptions.py` + `db.py` :** every SQL statement lives behind a small, named function, and the new `EmptyNoteError` names the one specific failure this step guards against. `find_notes` lowercases both sides of the comparison, so searching "Dentist" still matches a note saved as "dentist". **If not:** SQL would be scattered through `server.py` , and a case-sensitive search would miss obvious, expected matches.

```
# src/exceptions.py
# The project's named errors. MissingConfigError: a required setting is
# missing. EmptyNoteError: a note's text was empty or whitespace-only.
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass

class EmptyNoteError(Exception):     # new in Step 2
    """Raised when a note's text is empty or whitespace-only."""
    pass
```

```
# src/db.py
# All SQLite for the notes store. init_db() creates the table; insert_note()
# adds one and returns its id; find_notes() does a case-insensitive substring
# search.
import sqlite3
```

```
from datetime import datetime

def init_db(path):
    conn = sqlite3.connect(path)
    conn.execute(
        """CREATE TABLE IF NOT EXISTS notes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            text TEXT NOT NULL,
            created_at TEXT NOT NULL
        )"""
    )
    conn.commit()
    conn.close()

def insert_note(path, text):
    conn = sqlite3.connect(path)
    cursor = conn.execute(
        "INSERT INTO notes (text, created_at) VALUES (?, ?)",
        (text, datetime.now().isoformat()),
    )
    conn.commit()
    new_id = cursor.lastrowid
    conn.close()
    return new_id

def find_notes(path, query):
    conn = sqlite3.connect(path)
    rows = conn.execute(
        "SELECT text FROM notes WHERE LOWER(text) LIKE ?",
        (f"%{query.lower()}%",),
    ).fetchall()
    conn.close()
    found = []
    for row in rows:
        found.append(row[0])
    return found
```

Also create an empty `src/__init__.py` (already there from Step 1) — `db.py` sits directly in `src/`.

> Back to this step · Hint 1 · Hint 2 · Solution

## Solution — db-backed tools, with a clean error the client can read

**Changed from Step 1:** `add_note` now writes to SQLite and validates its input; `search_notes` is new. The in-memory list is gone.

**Story — `server.py`:** `add_note` raises `EmptyNoteError` on empty text, then **catches it itself and returns a clear error string** — because the official `mcp` SDK, when a tool raises, hands the client back only a generic *"Error executing tool add_note"*, with no useful reason attached. Catching it and returning `f"Error: {e}"` as the tool's normal result is Doc06's "get a tool's failure back to the model, correctly" pattern — a clean, readable result, not a crash and not a vague message. **If not:** the client would see a useless generic error, or `add_note("")` would silently store a blank note.

```
# server.py
# The MCPForge MCP server, Step 2: 2 Tools (add_note, search_notes) backed by
# SQLite. Run with `mcp dev server.py`. NEVER print() here or in anything it
# imports — on stdio, stdout is the protocol channel; logging goes to stderr.
from mcp.server.mcpserver import MCPServer

from src.config import load_config
from src.db import find_notes, init_db, insert_note     # new in Step 2
from src.exceptions import EmptyNoteError               # new in Step 2
from src.utils.logger import get_logger

config = load_config()
logger = get_logger(__name__)
DB_PATH = config.notes_db_path     # new in Step 2
init_db(DB_PATH)                   # new in Step 2

mcp = MCPServer("notes-server")

# changed in Step 2: db-backed, with content validation
@mcp.tool()
def add_note(text):
    """Add a new note to the local notes store. Use this whenever the user
    wants to save or jot down a piece of text. Rejects empty or
    whitespace-only text. Returns a short confirmation with the new id."""
    try:
        cleaned = text.strip()
        if not cleaned:
            raise EmptyNoteError("note text cannot be empty or whitespace.")
```

```
        new_id = insert_note(DB_PATH, cleaned)
        logger.info(f"Note {new_id} added")
        return f"Note added with id {new_id}."
    except EmptyNoteError as e:
        logger.warning("Rejected an empty note")
        return f"Error: {e}"

# new in Step 2
@mcp.tool()
def search_notes(query):
    """Search stored notes for a case-insensitive substring match. Returns
    the matching notes' text, or an empty list if nothing matches."""
    return find_notes(DB_PATH, query)

if __name__ == "__main__":
    logger.info("Starting notes-server over stdio")
    mcp.run(transport="stdio")
```

## The Step 2 tests

**Story —** `tests/unit/test_db.py` **:** this exercises the SQL helper functions directly against a throwaway test database, no server or client involved at all. **If not:** a search bug would look like a model mistake during a real run, instead of being caught here instantly.

```
# tests/unit/test_db.py
# Checks the SQLite helpers on a throwaway database. Each case prints OK or
# WRONG. No server, no network.
# Run from the repo root: python -m tests.unit.test_db
import os

from src.db import find_notes, init_db, insert_note

DB = "test_notes_tmp.db"


def fresh_db():
    if os.path.exists(DB):
        os.remove(DB)
    init_db(DB)


def test_insert_returns_id():
    fresh_db()
    new_id = insert_note(DB, "Buy milk and eggs")
    result = "OK"
```

```
    if new_id != 1:
        result = "WRONG"
    print(f"case 1 (insert returns id)       -> {result}")


def test_find_is_case_insensitive():
    fresh_db()
    insert_note(DB, "Call the dentist to reschedule")
    found = find_notes(DB, "DENTIST")
    result = "OK"
    if found != ["Call the dentist to reschedule"]:
        result = "WRONG"
    print(f"case 2 (case-insensitive search) -> {result}")


def test_find_no_match_is_empty():
    fresh_db()
    insert_note(DB, "Buy milk")
    result = "OK"
    if find_notes(DB, "dentist") != []:
        result = "WRONG"
    print(f"case 3 (no match -> empty)       -> {result}")
    if os.path.exists(DB):
        os.remove(DB)


if __name__ == "__main__":
    test_insert_returns_id()
    test_find_is_case_insensitive()
    test_find_no_match_is_empty()
```

**Changed from Step 1:** the integration test's `add_note` case now expects the `"id 1"` wording, and two cases are added — the case-insensitive search, and the empty-note rejection.

```
# tests/integration/test_server.py
# Launches server.py over stdio with a real MCP client and exercises what it
# exposes. Each case prints OK or WRONG. Needs the mcp SDK; no API key.
# Run from the repo root: python -m tests.integration.test_server
import asyncio
import os

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client


def show(case, passed):
```

```
    result = "OK"
    if not passed:
        result = "WRONG"
    print(f"{case} -> {result}")


async def main():
    if os.path.exists("notes.db"):     # new in Step 2: a clean store
        os.remove("notes.db")
    server_params = StdioServerParameters(command="python", args=["server.py"])
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            tools = await session.list_tools()
            names = []
            for t in tools.tools:
                names.append(t.name)
            show("case 1 (tools listed)", "add_note" in names
                 and "search_notes" in names)

            added = await session.call_tool("add_note", {"text": "Buy milk"})
            show("case 2 (add_note works)", "id 1" in added.content[0].text)

            # new in Step 2
            found = await session.call_tool("search_notes", {"query": "MILK"})
            show("case 3 (search, case-insensitive)",
                 "Buy milk" in found.content[0].text)

            # new in Step 2
            bad = await session.call_tool("add_note", {"text": "   "})
            show("case 4 (empty note rejected)",
                 bad.content[0].text.startswith("Error"))


if __name__ == "__main__":
    asyncio.run(main())
```

**Run it** (from the repo root):

```
python -m tests.unit.test_db
python -m tests.integration.test_server
```

**Prove persistence by hand:** add 2-3 notes through the Inspector, fully stop and restart `mcp dev server.py`, and `search_notes` still finds them — that's the real proof SQLite works, not just that the code ran once.

### Expected unit output:

```
case 1 (insert returns id)       -> OK
case 2 (case-insensitive search) -> OK
case 3 (no match -> empty)       -> OK
```